НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Московский институт электроники и математики им. А.Н. Тихонова

ОБЗОР НАУЧНОЙ ЛИТЕРАТУРЫ

Разработка методики тестирования процессоров архитектуры RISC-V на соответствие требованиям расширения ZKT

Соколов А.Д.

Москва 2025

СОДЕРЖАНИЕ

1 Введение..... 3
2 Теоретические основы и постановка проблемы..... 4
2.1. Атаки по временным каналам..... 4
2.2. Constant-time execution..... 4
2.3. Архитектурные предпосылки RISC-V и Zkt..... 4
3 Классификация существующих подходов..... 5
3.1. Архитектурные методы..... 5
3.2. Программные методы..... 5
3.3. Методы анализа и верификации..... 5
4 Обзор источников..... 6
4.1 Классические работы по timing-атакам..... 6
4.2 Constant-time на уровне алгоритмов и компиляторов..... 6
4.3 Микроархитектурные источники утечек..... 7
4.4 Архитектурные решения и RISC-V..... 8
4.5 Методы измерения и верификации timing..... 9
5 Анализ источников..... 11
5.1 Сравнение архитектурных и программных подходов..... 11
5.2 Анализ методов верификации..... 12
5.3 Применимость к требованиям Zkt..... 13
6 Заключение..... 14
СПИСОК ИСТОЧНИКОВ..... 16

1 Введение

С ростом распространения встраиваемых систем и криптографических приложений возрастает значимость защиты от атак по побочным каналам. Одним из наиболее распространенных и практически значимых классов таких атак являются атаки по временным характеристикам выполнения программ.

Архитектура RISC-V, получившая широкое распространение благодаря открытому исходному коду и модульному подходу к выстраиванию архитектуры, активно используется в системах, предъявляющих повышенные требования к безопасности. В связи с этим особый интерес представляет расширение Zkt, регламентирующее требования к независимости времени выполнения инструкций от обрабатываемых данных.

Целью данного обзора литературы является анализ существующих научных и технических подходов к обеспечению и верификации постоянства времени выполнения инструкций, а также выявление ключевых проблем и направлений дальнейших исследований в данной области.

2 Теоретические основы и постановка проблемы

2.1. Атаки по временным каналам

Атаки по временным каналам основаны на анализе зависимости времени выполнения вычислений от обрабатываемых данных. Впервые данный класс атак был систематически описан в работе П. Кочера, где было показано. что даже незначительные вариации времени выполнения могут привести к утечке секретной информации[1].

2.2. Constant-time execution

Под постоянством времени выполнения (constant-time execution) понимается свойство алгоритма или аппаратной реализации, при котором время выполнения не зависит от значений входных данных. Однако на практике обеспечение данного свойства осложняется микроархитектурными особенностями процессоров, такими как конвейеризация, кэш-память и предсказание переходов.

2.3. Архитектурные предпосылки RISC-V и Zkt

Архитектура RISC-V изначально проектировалась как модульная и расширяемая. В рамках обеспечения криптографической безопасности был предложен ряд расширений, включая Zkt [17], направленное на формализацию требований к независимости временных характеристик выполнения инструкций от данных.

3 Классификация существующих подходов

3.1. Архитектурные методы

В ряде работ предлагаются архитектурные методы обеспечения постоянства времени выполнения, основанные на ограничении или упрощении микроархитектурных механизмов. К таким методам относятся фиксированная латентность инструкций, отказ от динамического предсказания переходов и упрощение подсистемы памяти.

3.2. Программные методы

Программные методы ориентированы на разработку алгоритмов и реализаций, устойчивых к timing-атакам, за счет исключения ветвлений и обращений к памяти, зависящих от секретных данных. Однако данные методы не гарантируют защиту в условиях сложной микроархитектуры.

3.3. Методы анализа и верификации

Архитектурные методы обеспечивают более высокий уровень защиты, но сопровождаются снижением производительности. Программные методы менее инвазивны, однако не позволяют полностью устранить микроархитектурные утечки.

В современных работах все больше внимания уделяется методам верификации, направленным на выявление нарушений constant-time на уровне микроархитектуры [5-7, 26].

4 Обзор источников

В данном разделе представлен систематизированный обзор публикаций и технических материалов, сгруппированных по направлениям.

4.1 Классические работы по timing-атакам

Основы анализа атак по временным каналам были заложены в работах П. Кочера, в которых впервые было показано, что зависимость времени выполнения криптографических операций от обрабатываемых данных может приводить к практической утечке секретной информации [1]. В работе Kocher (1996) продемонстрировано, что даже при отсутствии прямого доступа к промежуточным значениям алгоритма измерения времени выполнения позволяют восстановить секретные параметры.

После этого в работе Bernstein представлены cache-timing атаки на реализацию AES-128, демонстрирующие влияние кэш-подсистемы на временные характеристики выполнения [2].

Работы Brumley и Boneh расширяют данный класс атак, демонстрируя возможность их проведения в удалённом сценарии [3].

Эти работы заложили основу для дальнейших исследований, посвящённых защите от временных утечек и формализации требований constant-time.

4.2 Constant-time на уровне алгоритмов и компиляторов

Большая часть исследований посвящена обеспечению свойства constant-time на уровне алгоритмов и программных реализаций. В ряде работ по constant-time программированию предлагаются практики [9-11], включающие отказ от ветвлений и обращений к памяти, зависящих от входных данных.

Последующие исследования показывают, что корректность constant-time программирования не всегда сохраняются после компиляции [9-11]. Работа Berthe et al. анализируют влияние оптимизаций компилятора на временные характеристики выполнения. Было выявлено, что оптимизации компилятора могут нарушить данное свойство.

В результате развивается направление формальной верификации constant-time свойств на уровне компилятора и промежуточных представлений, включая анализ SSA-форм [11] и применение подходов, связанных с формализацией информационной безопасности программ (например, через self-composition) [12]..

Таким образом, программные методы являются необходимыми, но недостаточными и требуют анализа на более низких уровнях абстракции.

4.3 Микроархитектурные источники утечек

Отдельный класс исследований посвящен анализу микроархитектурных механизмов, являющихся источниками временных утечек. В работах по анализу кэш-памяти показано, что даже при отсутствии явных зависимостей в коде, поведение кэш-подсистемы может приводить к измеримым различиям во времени выполнения [5].

Помимо кэш-памяти, источниками временных утечек являются конвейеризация, предсказание переходов и спекулятивное выполнение. Работы, посвященные атакам Spectre и Meltdown [6, 7], продемонстрорвали, что микроархитектурные оптимизации направленные на повышение производительности, могут приводить к утечкам информации, выходящим за рамки классической модели безопасности.

Данные исследования подчеркивают, что обеспечение constant-time свойств невозможно рассматривать исключительно на уровне программного кода и требует учета микроархитектурных особенностей [8, 14].

4.4 Архитектурные решения и RISC-V

Архитектурные методы направлены на ограничение микроархитектурных механизмов, способных приводить к временным утечкам. В отличие от программных подходов, данные методы предполагают внесение ограничений непосредственно в архитектуру процессорного ядра и, как следствие, потенциально обеспечивают более сильные гарантии безопасности [13].

Архитектура RISC-V предоставляет благоприятные предпосылки для реализации подобных методов благодаря своей модульности и формализованной спецификации. Базовый набор инструкций RISC-V определяет семантику выполнения без привязки к конкретной микроархитектуре, что позволяет реализовывать ядра с различными компромиссами между производительностью и безопасностью [15-17].

В рамках обеспечения криптографической устойчивости были предложены специализированные расширения архитектуры RISC-V, включая набор криптографических расширений и расширение Zkt [18, 19]. Расширение Zkt формализует требование независимости времени выполнения инструкций от обрабатываемых данных, вводя тем самым архитектурный контракт, который должен соблюдаться реализацией процессорного ядра.

В документации и технических отчетах по реализации открытых RISC-V ядер (таких как NEORV32 и CV32E40) [20, 21] рассматриваются различные подходы к достижению детерминированного времени

выполнения, включая фиксированную латентность инструкций, отказ от спекулятивного выполнения и упрощенную организацию конвейера. Эти решения, как правило, сопровождаются снижением производительности, однако позволяют существенно упростить анализ временных характеристик.

В контексте RISC-V архитектурные методы представляют собой компромисс между безопасностью и производительностью. Наличие формализованных требований в виде расширения Zkt делает возможным не только проектирование, но и последующую верификацию соответствия конкретных реализаций данным требованиям.

4.5 Методы измерения и верификации timing

Для подтверждения соответствия процессорных ядер требованиям постоянства времени выполнения необходимы формализованные методы измерения и верификации временных характеристик. В литературе представлены как аналитические, так и экспериментальные подходы, различающиеся уровнем абстракции и степенью достоверности получаемых результатов.

Одним из распространённых методов является моделирование с точностью до такта (cycle-accurate simulation) [24], позволяющее анализировать временное поведение инструкций на уровне архитектурного и микроархитектурного описания. Данный подход обеспечивает воспроизводимость результатов и удобен на этапе проектирования, однако его точность напрямую зависит от полноты и корректности модели.

Смежным направлением анализа временных характеристик является анализ наихудшего времени выполнения (Worst-Case Execution Time, WCET), широко применяемый в задачах верификации встроенных и real-time систем. В рамках WCET-анализа используются статические методы оценки времени

выполнения с учетом конвейерной структуры и кэш-памяти, что делает данные подходы методологически близкими к анализу требований constant-time [23].

Более детальный анализ возможен на уровне описания аппаратуры (RTL) [22], где используются трассировка сигналов и анализ временных диаграмм. RTL-уровень позволяет выявлять зависимости времени выполнения от данных, обусловленные логикой управления, конвейерными конфликтами и взаимодействием функциональных блоков. При этом сложность анализа существенно возрастает, особенно для нетривиальных ядер.

Экспериментальные методы верификации включают измерения на реальных аппаратных реализациях, в том числе с использованием FPGA-прототипов [25]. Такие методы позволяют учитывать реальные задержки и эффекты, не всегда отраженные в моделях, однако подвержены влиянию шумов и требуют статистической обработки результатов.

В ряде работ предлагается использование статистических тестов для выявления зависимости распределения времени выполнения от входных данных. Подобные методы позволяют формально оценивать вероятность нарушения требований constant-time, однако не дают строгих гарантий отсутствия утечек и требуют аккуратного выбора экспериментального дизайна.

Рассмотренные методы показывают, что верификация временных характеристик должна носить многоуровневый характер, сочетая моделирование, анализ аппаратного описания и экспериментальные измерения. Такой подход позволяет повысить достоверность выводов и выявлять нарушения требований Zkt на различных этапах жизненного цикла процессорного ядра [22-25].

5 Анализ источников

В данном разделе проводится сравнительный анализ рассмотренных источников с точки зрения применимости предложенных подходов к задаче обеспечения и верификации постоянства времени выполнения инструкций в процессорных ядрах архитектуры RISC-V. Особое внимание уделяется соответствию данных подходов требованиям расширения Zkt, а также выявлению ограничений и нерешенных проблем, обозначенных в литературе.

5.1 Сравнение архитектурных и программных подходов

Архитектурные и программные методы обеспечения constant-time ориентированы на разные уровни абстракции и имеют различные свойства. Архитектурные методы направлены на устранение или строгую регламентацию микроархитектурных механизмов, способных приводить к временным утечкам, за счет фиксированной латентности инструкций, упрощения конвейера и отказа от спекулятивного выполнения [13-16, 20-21].

Преимуществом архитектурных подходов является возможность формального задания требований к временным характеристикам выполнения и, как следствие, потенциальная возможность их строгой верификации. Однако данные методы, как правило, сопровождаются снижением производительности и гибкости архитектуры, что ограничивает их применимость в высокопроизводительных системах.

Программные методы, напротив, ориентированы на масштабируемость и переносимость и широко применяются на практике. Тем не менее, как показано в рассмотренных источниках, данные методы не обеспечивают

строгих гарантий безопасности в условиях сложной микроархитектуры и зависят от корректности компиляции и поведения аппаратной реализации [5-7, 9-11].

Таким образом, использование только программных методов недостаточно для выполнения требований Zkt и требует архитектурной поддержки и верификации.

5.2 Анализ методов верификации

Методы верификации временных характеристик, представленные в литературе, существенно различаются по уровню абстракции и степени достоверности получаемых результатов. Моделирование с точностью до такта широко используется на ранних этапах проектирования и позволяет выявлять потенциальные нарушения требований constant-time в архитектурных моделях. Однако данный подход зависит от полноты модели и не всегда отражает реальные задержки аппаратной реализации [24].

Анализ на уровне RTL предоставляет более детальное представление о временном поведении процессорного ядра и позволяет учитывать влияние логики управления и конвейерных конфликтов. В то же время сложность RTL-анализа и объём обрабатываемых данных существенно ограничивают его масштабируемость и применимость для сложных ядер [22].

Экспериментальные методы, основанные на измерениях на реальном оборудовании или FPGA-прототипах, позволяют учитывать реальные аппаратные эффекты, однако подвержены влиянию шумов и требуют статистической интерпретации результатов [25, 26]. В литературе подчеркивается, что статистические методы не обеспечивают строгих гарантий отсутствия утечек, а лишь позволяют оценить вероятность нарушения требований constant-time.

В совокупности рассмотренные работы указывают на необходимость многоуровневого подхода к верификации временных характеристик, сочетающего моделирование, анализ аппаратного описания и экспериментальные измерения.

5.3 Применимость к требованиям Zkt

Анализ источников показывает, что расширение Zkt формализует требования, которые не могут быть проверены исключительно на уровне программного кода. Большинство программных методов и статических анализаторов не обладают информацией о микроархитектурных особенностях реализации и, следовательно, не позволяют достоверно оценить соответствие требованиям Zkt [6-7, 9-11].

Архитектурные подходы, ориентированные на детерминированное время выполнения, являются наиболее согласованными с моделью Zkt, однако требуют наличия формализованных методик проверки соответствия конкретной реализации заявленным свойствам. При этом в литературе отсутствует единый подход к верификации требований Zkt, применимый к различным реализациям RISC-V ядер [18-21].

Таким образом, можно заключить, что существующие исследования формируют теоретическую и методическую базу для анализа временных характеристик, однако не предлагают универсальной и воспроизводимой методики тестирования процессоров архитектуры RISC-V на соответствие требованиям расширения Zkt. Данный пробел определяет актуальность дальнейших исследований в данном направлении.

6 Заключение

Проведенный обзор научной литературы показал, что атаки по временным каналам остаются одним из наиболее актуальных и практически значимых классов атак по побочным каналам, особенно в контексте криптографических вычислений. Несмотря на значительный объём исследований, направленных на разработку методов защиты, проблема обеспечения и верификации постоянства времени выполнения по-прежнему не имеет универсального решения.

Анализ существующих подходов продемонстрировал, что программные методы constant-time программирования являются необходимым, но недостаточным условием защиты, поскольку не учитывают микроархитектурные особенности конкретных процессорных реализаций. Архитектурные методы, напротив, позволяют формализовать требования к временным характеристикам выполнения инструкций, однако сопровождаются компромиссами в производительности и требуют специализированных механизмов верификации.

Архитектура RISC-V, благодаря своей модульности и формализованной спецификации, предоставляет благоприятную основу для реализации и анализа таких механизмов. Расширение Zkt вводит архитектурный контракт, регламентирующий независимость времени выполнения инструкций от обрабатываемых данных, тем самым создавая предпосылки для формальной оценки соответствия процессорных ядер требованиям constant-time.

Проведенный обзор показал, что в настоящее время отсутствует единая методика тестирования процессоров RISC-V на соответствие требованиям расширения Zkt. Существующие методы верификации, включая моделирование с точностью до такта, анализ на уровне RTL и экспериментальные измерения на аппаратных прототипах, рассматриваются в литературе преимущественно разрозненно и не образуют целостного подхода.

Таким образом, результаты обзора подтверждают актуальность разработки комплексной методики тестирования временных характеристик выполнения инструкций в процессорных ядрах RISC-V, сочетающей архитектурный анализ и многоуровневую верификацию. Сформулированные выводы могут быть использованы в качестве теоретической и методической основы для дальнейших исследований в данном направлении.

СПИСОК ИСТОЧНИКОВ

1. Kocher P. C. Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems [Электронный ресурс]. 1996. URL: https://paulkocher.com/doc/TimingAttacks.pdf
2. Bernstein D. J. Cache-Timing Attacks on AES [Электронный ресурс]. – 2005. – URL: https://cr.yp.to/antiforgery/cachetiming-20050414.pdf
3. Brumley D., Boneh D. Remote Timing Attacks Are Practical // 12th USENIX Security Symposium. 2003. URL: https://www.usenix.org/conference/12th-usenix-security-symposium/remote-timing-attacks-are-practical
4. Brumley D., Boneh D. Remote timing attacks are practical // Computer Networks. 2005. URL: https://www.sciencedirect.com/science/article/abs/pii/S1389128605000125
5. Yarom Y., Falkner K. Flush+Reload: A High Resolution, Low Noise, L3 Cache Side-Channel Attack // USENIX Security Symposium. 2014.
6. Kocher P., Horn J., Fogh A., et al. Spectre Attacks: Exploiting Speculative Execution // Communications of the ACM. 2020. DOI: 10.1145/3399742.
7. Lipp M., et al. Meltdown: Reading Kernel Memory from User Space // 27th USENIX Security Symposium. 2018. URL: https://www.usenix.org/conference/usenixsecurity18/presentation/lipp
8. Mangard S., Oswald E., Popp T. Power Analysis Attacks: Revealing the Secrets of Smart Cards. Berlin: Springer, 2007. 300 p.
9. Almeida J. B., et al. Verifying Constant-Time Implementations // 25th USENIX Security Symposium. 2016. URL: https://www.usenix.org/system/files/conference/usenixsecurity16/sec16_paper_almeida.pdf
10. Barthe G., Grégoire B., Laporte V. Secure Compilation of Side-Channel Countermeasures: The Case of Cryptographic “Constant-Time” // IEEE CSF. 2018. URL:

https://members.loria.fr/VLaporte/files/CSF2018_ConstantTimePreservation.pdf

11. Barthe G., Blazy S., Grégoire B., et al. Formal Verification of a Constant-Time Preserving C Compiler // Proc. ACM Program. Lang. (POPL). 2020. DOI: 10.1145/3371075.
12. Barthe G., D'Argenio P. R., Rezk T. Secure Information Flow by Self-Composition [Электронный ресурс]. PDF URL: https://www-sop.inria.fr/members/Tamara.Rezk/publication/Barthe-DArgenio-Rezk.pdf
13. Hennessy J., Patterson D. Computer Architecture: A Quantitative Approach. 6th ed. Morgan Kaufmann, 2019. 856 p.
14. Fog A. The microarchitecture of Intel, AMD and VIA CPUs [Электронный ресурс]. URL: https://www.agner.org/optimize/microarchitecture.pdf
15. Waterman A., Asanović K. (eds.) The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA. 2016. URL: https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-118.pdf
16. Waterman A., Asanović K. (eds.) The RISC-V Instruction Set Manual, Volume II: Privileged Architecture. 2016. URL: https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-161.pdf
17. RISC-V International. Ratified Specifications [Электронный ресурс]. URL: https://riscv.org/specifications/ratified/
18. RISC-V Unified Database / ISA Manual (draft). Zkt Extension [Электронный ресурс]. URL: https://riscv-software-src.github.io/riscv-unified-db/manual/html/isa/isa_20240411/exts/Zkt.html
19. RISC-V International. RISC-V Cryptography Extensions Specification [Электронный ресурс]. URL: https://github.com/riscv/riscv-crypto
20. Nolting S. NEORV32 Processor Documentation [Электронный ресурс]. – URL: https://github.com/stnolting/neorv32

21. OpenHW Group. CV32E40 Core Documentation [Электронный ресурс]. – URL: https://docs.openhwgroup.org/projects/cv32e40p-user-manual/
22. Doychev G., Feld D., Köpf B., Mauborgne L., Reineke J. CacheAudit: A Tool for the Static Analysis of Cache Side Channels // 22nd USENIX Security Symposium. 2013. PDF: https://www.usenix.org/system/files/conference/usenixsecurity13/sec13-paper_doychev.pdf
23. Wilhelm R., Engblom J., Ermedahl A., et al. The worst-case execution-time problem: overview of methods and survey of tools // ACM Transactions on Embedded Computing Systems. 2008. DOI: 10.1145/1347375.1347389.
24. Binkert N., Beckmann B., Black G., et al. The gem5 simulator // ACM SIGARCH Computer Architecture News. 2011. DOI: 10.1145/2024716.2024718. PDF: https://people.csail.mit.edu/tushar/papers/pdfs/gem5_can2011.pdf
25. Zhao X., et al. FPGA-Based Remote Power Side-Channel Attacks [Электронный ресурс]. 2018. URL: https://cpb-us-w2.wpmucdn.com/sites.coecis.cornell.edu/dist/7/89/files/2018/04/SP2018-FPGA-2m12dnp.pdf
26. Lou X., et al. A Survey of Microarchitectural Side-channel Vulnerabilities, Attacks and Defenses in Cryptography [Электронный ресурс]. 2021. URL: https://arxiv.org/abs/2103.14244